

Chapter 7

Working with Text Data

Bag-of-words, tf-idf, n-grams,
and topic modeling

Learning Objectives

- Represent text numerically with the **bag-of-words** model
- Trim and reweight features with **stopwords**, `min_df`, and **tf-idf**
- Capture word order with **n-grams**
- Normalize words via **stemming** and **lemmatization**
- Discover themes unsupervised with **LDA** topic modeling

Types of Data Represented as Strings

Not every string is "text" — four kinds to tell apart:

- **Categorical** — a value from a fixed list (colors from a menu)
- **Free strings mapped to categories** — messy user input for a category
- **Structured strings** — addresses, dates, phone numbers
- **Text** — free-form sentences (our focus)

A collection of text documents is a **corpus**; each data point is a **document**.
(*Terms from information retrieval and NLP.*)

Example: Sentiment Analysis of Movie Reviews

The chapter's running task: classify IMDb reviews as **positive** or **negative**.

- Stanford dataset (Andrew Maas); ratings $\geq 7 \rightarrow$ positive, $\leq 4 \rightarrow$ negative
- **25,000** training + **25,000** test documents, perfectly **balanced** (12,500 each)
- A standard **binary text classification** problem

We must turn variable-length text into fixed-length numeric vectors.

Loading the Data

```
from sklearn.datasets import load_files
reviews_train = load_files("data/aclImdb/train/")
text_train, y_train = reviews_train.data, reviews_train.target

# clean out HTML line breaks
text_train = [doc.replace(b"<br />", b" ") for doc in text_train]
```

`load_files` reads a folder-per-class layout (`pos/` , `neg/`) into texts + labels.

Always **inspect and clean** raw text before modeling.

Part 1

The Bag-of-Words Model

The Bag-of-Words Idea

Discard structure (order, grammar) — keep only **word counts**.

Three steps:

1. **Tokenization** — split each document into words (tokens)
2. **Vocabulary building** — collect all words across the corpus, number them
3. **Encoding** — count how often each vocabulary word appears per document

Word **order is thrown away** — only counts remain.

The Bag-of-Words Pipeline

Worked on the sentence **"This is how you get ants."**

1. **Tokenize** → `['this', 'is', 'how', 'you', 'get', 'ants']`

2. **Build vocabulary** over *all* documents (alphabetical) →

`['aardvark', 'amsterdam', 'ants', ... 'you', 'your', 'zyxst']`

3. **Encode** → a count over the whole vocabulary →

`[0, ..., 1 (ants), ..., 1 (get), ..., 1 (you), ..., 0]`

Only present words get a nonzero count — everything else is **0** (hence sparse).

CountVectorizer on a Toy Dataset

```
from sklearn.feature_extraction.text import CountVectorizer
bards_words = ["The fool doth think he is wise,",
               "but the wise man knows himself to be a fool"]
vect = CountVectorizer().fit(bards_words)
len(vect.vocabulary_)           # 13 words
bag = vect.transform(bards_words) # <2x13 sparse matrix>
bag.toarray()                   # dense counts (0/1 here)
```

CountVectorizer is a **transformer**: `fit` builds the vocabulary, `transform` encodes.

Bag-of-Words for Movie Reviews

```
vect = CountVectorizer().fit(text_train)
X_train = vect.transform(text_train)
X_train.shape      # (25000, 74849)
```

- **74,849** features — one per unique word in the vocabulary
- Stored as a **sparse matrix** (each review uses few words) — a dense version would exhaust memory

Inspecting the Vocabulary

```
feature_names = vect.get_feature_names()
```

- The **first ~10** entries are **numbers** ("00", "007", ...) that appear in reviews
- Singular & plural are **separate** words: "*drawback*" vs "*drawbacks*"
- Many tokens are rare or uninformative

The raw vocabulary is noisy — the next steps clean it up.

A First Classifier

High-dimensional sparse counts → a **linear model** is a strong choice.

```
from sklearn.linear_model import LogisticRegression
cross_val_score(LogisticRegression(), X_train, y_train, cv=5).mean()    # 0.88

grid = GridSearchCV(LogisticRegression(), {'C': [0.001, 0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_train, y_train)        # best C = 0.1, CV 0.89
grid.score(vect.transform(text_test), y_test)    # test 0.88
```

min_df: Dropping Rare Words

Require a token to appear in at least N documents.

```
vect = CountVectorizer(min_df=5).fit(text_train)    # ≥5 documents  
vect.transform(text_train).shape                  # (25000, 27271)
```

- **74,849** → **27,271** features — about a third
- A word in only one document can't help on the test set
- Accuracy holds at **0.89** — faster and more interpretable

Stopwords

Very frequent words (*the, and, is*) carry little signal.

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS    # 318 words
CountVectorizer(min_df=5, stop_words="english")    # 27271 → 26966
```

- Two approaches: a **fixed stopwords list**, or drop very frequent tokens with `max_df`
- Here it removed only 305 words and **slightly hurt** accuracy
- Fixed lists mainly help **small** datasets

Part 2

tf-idf & Model Inspection

Rescaling with tf-idf

Term frequency–inverse document frequency weights words by how *informative* they are.

- **High** weight: frequent in **this** document, but rare across the corpus
- **Low** weight: common everywhere (or only in a few very long documents)

Two classes: `TfidfTransformer` (on counts) and `TfidfVectorizer` (from raw text).

Both apply **L2 normalization** so document length doesn't matter.

The tf-idf Formula

The score for word w in document d (as in scikit-learn):

$$\text{tfidf}(w, d) = \text{tf} \cdot \left(\log \frac{N + 1}{N_w + 1} + 1 \right)$$

- **tf** — how often w appears in d (term frequency)
- **N** — total documents; **N_w** — documents containing w
- Rare words → large `log` term → **higher** weight
- Then **L2-normalize** each document → length doesn't affect the vector

(You don't need to memorize it to use `TfidfVectorizer`.)

tf-idf in Practice

tf-idf uses corpus statistics → wrap it in a **Pipeline** to avoid leakage.

```
from sklearn.feature_extraction.text import TfidfVectorizer
pipe = make_pipeline(TfidfVectorizer(min_df=5),
                    LogisticRegression())
grid = GridSearchCV(pipe, {'logisticregression__C': [0.001, 0.01, 0.1, 1, 10]}, cv=5)
grid.fit(text_train, y_train)      # CV improves to ~0.89
```

What tf-idf Finds

- **Low** tf-idf → words common across documents: mostly **stopwords** (*the, no*)
 - Interestingly, *good, great, bad* also score low — very frequent overall
- **High** tf-idf → **show/franchise-specific** words (*pokemon, smallville*)
 - Distinctive, but not always useful for **sentiment**

*tf-idf is **unsupervised** — “important” means distinctive, not predictive of the label.*

Investigating Model Coefficients

A linear model over bag-of-words has **one coefficient per word** → highly interpretable.

```
grid.best_estimator_.named_steps["logisticregression"].coef_
```

- Largest **negative** coefficients → *worst, waste, disappointment, laughable*
- Largest **positive** coefficients → *excellent, wonderful, enjoyable, refreshing*
- A quick sanity check that the model learned something sensible

Part 3

Beyond Single Words

Bag-of-Words with n-Grams

Single words (unigrams) lose context:

*"it's bad, not good at all" and "it's good, not bad at all" get the **same** representation.*

n-grams = sequences of n adjacent tokens, set via `ngram_range=(min, max)` :

```
CountVectorizer(ngram_range=(1, 1))    # unigrams (default)
CountVectorizer(ngram_range=(2, 2))    # bigrams only
CountVectorizer(ngram_range=(1, 3))    # uni + bi + trigrams
```

Choosing the n-Gram Range

- Bigrams usually help; trigrams add a little more
- But features **explode**: potentially unigrams², unigrams³ ...
- Tune the range with grid search

On IMDB (`TfidfVectorizer` + logistic regression):

ngram_range	best CV
(1, 1)	0.89
(1, 2)	0.906
(1, 3)	0.91 (C = 100)

What n-Grams Capture

With bigrams / trigrams, **context** appears in the top coefficients:

- "*not worth*" → strongly **negative**
- "*definitely worth*", "*well worth*" → strongly **positive**

These flip the meaning of "*worth*" — impossible to capture with unigrams alone.
Most trigrams are common phrases, so their individual impact is small.

Advanced Tokenization

Normalize related word forms so they share one feature.

Stemming

- Rule-based chopping (Porter, `nltk`)
- Crude: *was* → *wa*, *worse* → *wors*

Lemmatization

- Dictionary + word role (`spacy`)
- Accurate: *was* → *be*, *worse* → *bad*

Both are forms of **normalization** that shrink the vocabulary.

Stemming vs Lemmatization: Example

Sentence: *"Our meeting today was worse than yesterday..."*

Word	Stemming (Porter)	Lemmatization (spaCy)
was	wa	be
worse	wors	bad
meeting (noun)	meet	meeting
meeting (verb)	meet	meet

Lemmatization uses the word's **role in the sentence** → the noun "meeting" stays, the verb "meeting" becomes "meet". Stemming can't tell them apart.

Custom Tokenizer & Its Effect

Pass your own tokenizer to `CountVectorizer(tokenizer=...)` .

```
lemma_vect = CountVectorizer(tokenizer=custom_tokenizer, min_df=5)
X_train_lemma = lemma_vect.fit_transform(text_train)    # 27271 → 21596 features
```

- Lemmatization acts like **regularization** (fewer, merged features)
- Biggest gains on **small** data: with only 1% for training, standard **0.721** → **0.731** with lemmatization

Part 4

Topic Modeling with LDA

Topic Modeling & LDA

Topic modeling = assign each document to one or more **topics**, unsupervised.

- **Latent Dirichlet Allocation (LDA)** is the most common method
- Finds groups of words that **frequently appear together**
- Each document is a **mixture** of topics; each topic a distribution over words
- Related to **NMF** decomposition (Chapter 3)

⚠ An LDA "topic" may not match a human topic — just co-occurring words.

LDA on Movie Reviews

```
vect = CountVectorizer(max_features=10000, max_df=.15)    # drop very common words
X = vect.fit_transform(text_train)

from sklearn.decomposition import LatentDirichletAllocation
lda = LatentDirichletAllocation(n_components=10, learning_method="batch",
                               max_iter=25, random_state=0)

document_topics = lda.fit_transform(X)
lda.components_.shape      # (10, 10000): word importance per topic
```

Inspect each topic by its **highest-weight words**.

Interpreting Topics

- **10 topics** → broad themes (war movies, comedies, TV series, children's films)
- **100 topics** → specific themes (music, thrillers, a "worst reviews" topic)
- LDA mostly discovers **genre-specific** and **rating-specific** groups of words

Topics have **no natural order**, like NMF components.

Using & Caveats of LDA

- Great for **understanding** and organizing a large corpus with no labels
- The `document_topics` matrix is a **compact representation** — useful for supervised learning when data is scarce
- ⚠ LDA is **randomized**: `random_state` changes the results
- Treat any conclusions **cautiously** — verify by reading actual documents

Where to Go Next in NLP

`CountVectorizer` and `TfidfVectorizer` are deliberately **simple**. For more:

- **Libraries:** `spacy` (fast, modern), `nltk` (comprehensive), `gensim` (topic models)
- **Word embeddings** (`word2vec`) — represent words as dense vectors capturing meaning, so “*king*” and “*queen*” sit close
- **Recurrent neural networks (RNNs)** — produce text as output → translation, summarization

Bag-of-words remains a strong, interpretable **baseline** to start from.

Chapter Summary

- **Bag-of-words** turns text into count vectors (word order discarded)
- Trim with `min_df` / **stopwords**; reweight with **tf-idf**
- **n-grams** recover context (*"not worth"*) at a feature-count cost
- **Stemming / lemmatization** normalize word forms
- Linear-model **coefficients** make text classifiers interpretable
- **LDA** discovers latent topics unsupervised

Next: Chapter 8 — Wrapping Up

End of Chapter 7

Representation is everything in NLP